

Using an ethnographic process to conduct requirements analysis for agile systems development

Nanda C. Surendra

Published online: 20 July 2007
© Springer Science+Business Media, LLC 2007

Abstract Since the late 1990s, agile or lightweight software development methodologies have experienced a meteoric rise of interest among systems developers who consider traditional systems development methodologies (such as Structured Systems Development, Information Engineering, and Rational Unified Process) too inflexible for building systems having unpredictable and changing requirements. Agile methodologies try to adapt to rapid requirement changes by having short, iterative development cycles and by encouraging frequent, open communication with customers. However, agile methodologies are still in the process of devising systematic approaches on how to be adaptive to unpredictable and changing requirements and how to incorporate customers into the systems development process. In this study, we show how an ethnographic research process called Strip Resolution can be applied as a systematic complement to projects applying agile development principles. We illustrate, in detail, how the Strip Resolution Process (SRP) enabled us, in the role of systems developers, to adapt to changing and unpredictable user requirements and to incorporate customers into the systems development process.

Keywords Agile methodologies · Systems development · Ethnography · Strip resolution process

Systems development has moved from no methodology, to “heavy” methodologies, to “lightweight” or agile

methodologies [12]. The early computer applications of the 1960s and 1970s were characterized by the phrase “code and fix.” Programmers wrote programs individualistically with little or no planning. They did not have any systematic way to determine if a computer application met user requirements or if they had eliminated the many bugs that occurred. The “code and fix” approach to software development was clearly untenable [6].

The alternative to the “code and fix” approach is the use of methodologies [22]. A methodology is a recommended collection of procedures, rules, techniques, and philosophy (the underpinning set of beliefs and assumptions) used to develop a software system [6]. The role of methodologies is to impose a disciplined process upon software development. The goal of methodologies is to make software development more predictable and efficient [12]. From the mid-1970s through the 1990s, several systems development methodologies emerged. Among these methodologies were the process-oriented Structured Systems Development, the data-oriented Information Engineering, and the object-oriented Rational Unified Process [13]. These methodologies were based on the systematic design and construction approaches used in the engineering disciplines. The use of the term “software engineering” for systematic approaches to systems development shows the influence of the engineering disciplines on these methodologies which have a mechanistic view of the systems development process [22]. Unlike engineers who construct artifacts such as buildings and vehicles having relatively predictable requirements, systems developers need to construct software applications that have rapidly changing user requirements. These systems development methodologies are too inflexible in developing systems with rapidly changing user requirements due to the following limitations:

N. C. Surendra (✉)
College of Business and Economics, West Virginia University,
1601 University Avenue, Morgantown, WV 26506, USA
e-mail: Nanda.Surendra@mail.wvu.edu

- (a) focus on documentation that forces systems developers to create a huge amount of unproductive paper work at the expense of using that time to develop effective software that meet user requirements [3, 5, 7, 9, 15],
- (b) emphasis on the process prescribed by the methodology and treating the people who develop the system as easily replaceable “plug-and-play” commodities [3, 5, 7, 9, 15],
- (c) treating user requirements as being a “fixed” set (rather than an emergent, changing set) that can be “captured” during the requirements analysis phase of the systems development process and kept “static” during the rest of the systems development process [3, 5, 7, 9, 15], and
- (d) long systems development cycles that are not conducive to rapid development needs of customers, especially since the advent of web applications over the past decade [22].

The inflexibility of these methodologies combined with their elaborate and complex processes have led to them being called “heavy” methodologies [12].

To counter the inflexibility of the “heavy” methodologies, a group of methodologies termed “agile” or “lightweight” software development methodologies have gained prominence since 2001 [2]. Extreme Programming or XP [7], Scrum [25], Crystal software methodologies [10], and Adaptive Software Development [15] are among the better known and more widely used agile methodologies. Agile methodologies try to adapt to rapid requirement changes by having short, iterative development cycles and by encouraging frequent, open communication with customers. However, these methodologies are still in the process of devising systematic approaches on how to be adaptive to unpredictable and changing requirements and how to incorporate customers into the systems development process. Alistair Cockburn, a prominent agile development methodologist, states that agile methodologists should draw upon the work of sociologists and anthropologists to develop a vocabulary for understanding the “soft” parts of systems development since they have been doing research on understanding people and communities for several decades [9]. Harvey and Myers [14, p. 13], state that “there is a gap between the process of knowledge generation conducted by researchers and that conducted by practitioners.” They suggest that ethnographic principles can be used to bridge this gap.

In this study, we act on the suggestions of authors coming from two different perspectives—Cockburn, an agile systems development practitioner and Harvey and Myers, ethnographic researchers in Information Systems—to illustrate how an ethnographic research process

called the Strip Resolution Process (SRP) [1] can be used as a systematic complement to projects applying agile development principles.

In the next section, we provide an overview of agile systems development methodologies and their underlying assumptions and philosophies. We discuss the ethnographic research approach used in this study in the second section. In the third section, we discuss the data collection methods used in our study and provide details of the organizational context in which this research was conducted. The agile characteristics of our development process are discussed in the fourth section. We describe the SRP in the fifth section. In the sixth section, we illustrate in detail how we used the SRP for incorporating users into the systems development process, for eliciting their requirements, and for adapting to changes in those requirements. In the seventh section, we discuss (a) the SRP serving as a bridge across the systems developer—user understanding gap, (b) the replicability of an interpretation using the SRP, (c) determining the validity of an interpretation using the SRP, (d) how the SRP compares to other reflective learning processes, and (e) limitations of our study. We provide some concluding thoughts in the eighth section.

1 Agile systems development methodologies

Agile or lightweight software development methodologies have experienced a “meteoric rise of interest” among systems developers [2] due to the shortcomings of the heavy methodologies combined with the increasing awareness of agile methodologies among systems developers. Extreme Programming or XP [5, 7], Scrum [25], Crystal software methodologies [9, 10], and Adaptive Software Development [15] are among the better known agile software development methodologies [9].

The emphasis of most agile methodologies is on “soft” aspects of systems development such as communication, collaboration, and feedback rather than the technical aspects of systems development such as technology, process, tool, notation, and modeling. Agile methodologies emphasize these soft aspects since they have been created as a reaction to traditional or heavy methodologies [7, 12, 22]. This does not mean that agile methodologies totally ignore the technical aspects; they do address them but with less emphasis compared to the soft aspects. Agile systems methodologies operate on the following three key underlying assumptions [27]:

- (a) *It does not cost significantly more to add or make changes to the system later compared to making those additions or changes now.* This assumption leads agile methodologies to recommend creating the

simplest possible design that would meet current user requirements. If the need arises later to add or make changes to the system, the agile developer makes the addition or change at that time. A technical factor that has contributed to, and to an extent enabled, this assumption is the extensive use of object-oriented and component-based design in agile development.

- (b) *Well written code is self-documenting and is more useful than extensive documentation.* Agile methodologies make this assumption based on two factors: (a) the difficulty of keeping documentation synchronized with code that is constantly changing leading to unusable and unused documentation and (b) code generates functionalities immediately usable by customers and, hence, is of more value to customers than documentation about the system.
- (c) *Face-to-face communication and extensive customer interaction are needed to determine user requirements.* Agile methodologies make this assumption based on two factors: (a) customers are the best judges of whether a system meets requirements and (b) face-to-face communication enables lack of, or incorrect, understanding to be detected and corrected faster compared to communication over a distance (such as by phone, e-mail, or video conferencing).

These assumptions have worked in the case of several projects that have used agile systems development methodologies [9, 15, 25]. Most of the documented successful projects have involved development teams of between 10 and 100 people co-located in a single site [9, 25]. However, an agile systems development methodology may not be appropriate for all projects [11, 27]. The underlying assumptions of agile methodologies may not hold or be appropriate for some projects. The assumption of extensive face-to-face communication with customers may not be applicable when customers' time is scarce, expensive, or needed for core business functionalities. The assumption that it does not cost significantly more to add or make changes to the system later may not be applicable for systems that need high safety or security (such as medical and health care systems). The assumption that well-written code is self-documenting may not be applicable in very large, enterprise systems where it might not be feasible to scan through several thousands of lines of code to find a particular functionality or module. These assumptions are not always explicitly stated by agile methodologists [27]. So, systems developers need to be aware of these underlying assumptions if they plan on using an agile methodology for their projects. If these assumptions don't hold in a specific project, then an agile systems development methodology would not be appropriate for that project.

2 Research method

Ethnography is the social research style that emphasizes encountering and making sense of alien worlds [1]. It consists of making sense of social action in one world from the perspective of an observer who is from another world. "Ethnographers immerse themselves in the life of the people they study and seek to place the phenomena studied in their social and cultural context" [22, p. 4].

Ethnography emphasizes understanding of situations rather than prediction. A good ethnography is one which sensitizes the reader "to the beliefs, values, and practices of the natives in another society. If, after reading the ethnography, actions which were previously seen as absurd, strange or irrational 'made sense', then that ethnography has achieved its purpose" [14, p. 18]. Since ethnography requires studying the phenomena of interest in its context, applying ethnography in studying Information Systems moves the Information Systems researcher closer to the context in which the systems are being used or developed. So, ethnography has the potential to move Information Systems researchers and practitioners closer together [14].

Over the past two decades, several Information Systems researchers have conducted ethnographic research studies. Their work informed our data collection and interpretation process. Zuboff's book published in 1988 is one of the first and best known ethnographic studies in the field of Information Systems [29]. Her book is considered among the most insightful studies on social and organizational phenomena related to the use of information technology. Orlikowski [23] used ethnography to explore the introduction of Computer Assisted Software Engineering (CASE) tools in the systems development process and found that these tools changed the relationships between functional and technical members of development teams in very fundamental and unproductive ways. Myers and Young [21] used critical ethnography to unearth how hidden agendas, power, and managerial assumptions become deeply embedded within Information Systems development projects. Schultze [24] used a form of ethnographic representation known as confessional ethnographic writing to understand knowledge management technologies.

Trauth and Jessup [26] conducted their research using both experimental and ethnographic research approaches to study Group Support System use. The specific ethnographic research approach they used was Agar's SRP. The "strips" (data or phenomena) they used for deriving their interpretation were transcripts of discussions between users of a Group Support System. The Trauth and Jessup work serves as an exemplar for our research in how to apply the SRP. In our study, we use multiple data sources for strips (observation of subjects' work practices in their everyday

work environment, semi-structured and unstructured, open-ended interviews of subjects, and documents used and generated by the subjects) compared to the single source of data (transcripts) used for strips by Trauth and Jessup. Such use of multiple sources of data allows the researcher to better validate his interpretations—a process that Yin [28, p. 99] calls “triangulating from multiple sources of evidence.”

Hughes, et al., [17] and Holzblatt and Beyer [16] have used ethnography for systems development, in contrast to Information Systems academic research. They suggest that requirements analysis be conducted with users in their own context, i.e., in their everyday work environment as they are in the processing of accomplishing their work. Such in-context requirements analysis allows systems developers to unearth the “taken-for-granted” and tacit work practices that users may not be able to articulate.

There are three main schools of ethnography [20]. Ethnographers of the “holistic” school focus on empathizing with the subjects being observed and, hence, require the ethnographer to “go native” and live with and like their subjects. The “thick description” school of ethnography focuses on searching for and analyzing symbolic forms of words, images, and behaviors with respect to one another and to the whole they comprise. The “critical ethnography” school suggests a critical dialog between the ethnographer and the subjects, with the ethnographer questioning and unearthing underlying assumptions and hidden agendas. The ethnography emerges from this critical dialog. Critical ethnography draws upon ideas from hermeneutics. Hermeneutics is the analysis of the meaning of a text and a major idea it uses is that of a hermeneutic circle, “the dialectic between the understanding of the text as a whole and the interpretation of its parts” [21 p. 19].

Agar introduced the SRP, which combines elements of the “thick description” and “critical ethnography” schools, to provide a language and a process that can help ethnographers both conduct and evaluate ethnographic research. The SRP is a relatively systematic step-by-step and easy to use process compared to other ethnographic approaches that can take several years to learn and use. However, true to its interpretive philosophy, it is not a mechanistic process. Like critical ethnography, it involves a critical dialog between the ethnographer and the subjects. It requires the ethnographer to engage in reflection when he observes actions, behavior, or situations that he does not understand. Though the SRP has similarities to other ethnographic approaches and hermeneutics, its’ relatively step-by-step and systematic process and its language are particularly helpful to beginning ethnographic researchers. This is why we consider the SRP as the most appropriate ethnographic process for use by systems developers. In this study, we use the SRP both in the role of researcher and in the role of

systems developer. The emphasis will be on discussing our use of the SRP in the role of systems developer.

3 Data collection

This study was conducted in the financial analysis department (StockAnalysisDepartment, a fictional name) of a Wall Street based Fortune 100 financial company over an 18-month period. When this study was initiated, StockAnalysisDepartment had 58 employees consisting of 45 stock analysts, 12 managers of stock analysts (each manager, depending on experience, managed between 3 and 5 analysts), and one department head. StockAnalysisDepartment’s clients were large corporations that had stock listings on the New York and London Stock Exchanges. Each client provided a list of companies that it considered its top ten competitors to StockAnalysisDepartment. The stock analysts tracked and compared investment management companies’ investments in the client and its list of ten competitors (how much of the client’s and competitors’ stock did an investment management company hold). They researched and categorized the investment management companies’ investment strategies (such as Growth, Value, Indexed, Quantitative, Theme, and Contrarian). If they found that some investment management companies were investing in the competitors’ stock but were not investing in the client’s stock, then, they prepared a report for the client stating the differential amount invested by the investment management company in the competitor and the investment strategy used by that company. This report enabled the client to target that investment management company with specific data and promotional material that might convince the investment manager to invest in their stock.

This research study revolved around a project for designing and developing a web application that could support the work flow and decision making of the company’s stock analysts. We worked on this project in the dual roles of researcher (one researcher, the author) and systems developer (two systems developers, the author and his systems development partner). As systems developers, the two of us worked through the entire systems development cycle of analysis, design, development, and implementation.

We used data collection techniques characteristic of ethnographic research [1, 19, 20, 28]: (a) observation of subjects in their work place, (b) based on our observations, discussion with subjects as they performed their work, (c) semi-structured (with a set of prepared questions) and unstructured, open-ended interviews, and (d) study of documents used by subjects for their work and created as a result of their work such as reports and financial analysis

spread sheets. We observed the work and decision flow of stock analysts. We conducted interviews with users at all levels of the organizational hierarchy. Among the users we interviewed were (a) five stock analysts who were the primary end-users of the application, (b) two managers of stock analysts—we call them “manager analysts”—who were secondary end-users of the application; each manager managed a group of between three and five stock analysts, and (c) the head of the department in which the system was implemented and who was one of the two sponsors of the project (the second sponsor being a manager analyst). We used the ethnographic process of Strip Resolution [1], explained in detail in Sect. 5, to better understand, integrate, and interpret the data we were collecting in the form of observation, interviews, and documents.

4 Agile systems development characteristics that apply to this project

Our project had the following characteristics of agile systems development [7, 27]:

- (a) **Pair Programming:** Two of us worked as Systems Developers on this project and we used the Pair Programming technique. We worked together on one computer. Each of us took turns controlling the keyboard while the other provided suggestions and feedback on the effectiveness of the logical design and the efficiencies of the code.
- (b) **Frequent communication with customer:** We frequently communicated with the two major project sponsors (the department head and a manager analyst) and the primary end users (stock analysts). We met with the project sponsors on a once-in-2-weeks basis and with the stock analysts on a once-a-month basis for most of the length of the project.
- (c) **Short, iterative development cycles:** At each fortnightly meeting with the project sponsors and monthly meeting with the stock analysts, we demonstrated functionalities that we had developed (especially the new or changed functionalities) and discussed whether they met their requirements. The customers gave detailed feedback (whether requirements were met or not; if not, why not) that we incorporated into our next system development iteration. When a functionality met all requirements and tests, we released the code module containing that functionality into production to be used by the stock analysts.
- (d) **Testing as a part of development:** For each code unit that we developed (the basic code units were stored procedures and triggers in the database and class methods in the business logic and data access layers

of our application), we created unit tests. On successful testing of a unit, we integrated that unit into the application and tested the application as a whole.

- (e) **Coding standards:** We were a small team of two developers and used common naming and design standards.

5 The Strip Resolution Process (SRP)

Strip Resolution is an ethnographic process. Agar introduced the SRP to provide a language and a process that can help guide ethnographic researchers [1]. It is a relatively systematic, step-by-step interpretation process compared to other ethnographic approaches. As a result, this process is better suited than any other ethnographic approach for those with little to no formal training in anthropological research methods—which is likely the case with the majority of systems developers.

The SRP moves through three stages: (a) from breakdowns of established understandings and taken-for-granted assumptions, (b) through a resolution of such breakdowns, and (c) to the construction of a coherent narrative [1]. The SRP is made up of the following five elements:

- (a) A *schema* is a conceptual abstraction that serves as the basis for “human information processing, e.g., perception and comprehension, categorization and planning, recognition and recall, and problem-solving and decision-making” [8, p. 430]. A schema is the combination of the interpreter’s frame of reference, goals, and plans at a given time. In simpler terms, a schema is the interpreter’s world view [26].
- (b) A *strip* is data that is produced from the interpreter’s observations, discussions with subjects, interviews of the subjects, participation in activities with the subjects, or study of documents. A strip is any phenomenon against which the interpreter tests his understanding (which is based on the interpreter’s existing schema).
- (c) A *breakdown* occurs when the interpreter applies his current schema to a strip and finds that he cannot understand the strip.
- (d) A *resolution* of a breakdown is the process by which the existing broken down schema is changed to develop a new schema that results in the interpreter understanding the strip.
- (e) A schema is *coherent* if it can be applied to many related strips without a resulting break down in the interpreter’s understanding of the phenomena represented by the strips.

Figure 1 illustrates a generic SRP. Schema-1, the interpreter’s first schema (based on his pre-existing

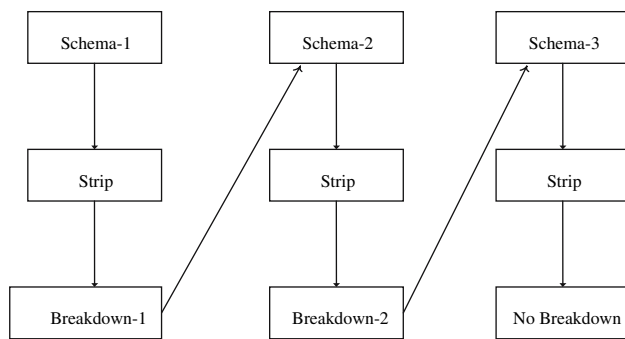


Fig. 1 The strip resolution process (adapted from Agar [1])

knowledge, research plans, research theory, and research goals), is applied to a strip (which is any source of data—observations, interviews, or documents—the interpreter has collected). In Fig. 1, the interpreter, using Schema-1, does not understand the strip leading to a breakdown in his schema (Breakdown-1). He modifies his Schema-1 to Schema-2 based on (a) his reflections on why he did not understand the strip and (b) new collection of data (which can be in the form of discussions, interviews, observation or participant-observation). He applies the modified schema on the same strip. He continues to have problems understanding the strip, leading to Breakdown-2. After further reflection and incorporating lessons learned from analyzing why his understanding broke down, he changes his schema from Schema-2 to Schema-3 and applies Schema-3 to the strip. Since he understands the strip now, there is no breakdown and resolution occurs. Schema-3 will have to be tested on other related strips to determine if it is a coherent schema. If none of the tests fail, then the schema is coherent. A coherent schema is able to explain other phenomena related to this strip. The SRP helps the interpreter attain the new or different frames of reference and knowledge that makes up his current Schema-3 compared to his initial Schema-1. Schema-3 enables the observer better understand and explain the situation compared to Schema-1.

We adapted Agar's SRP for systems development. In object-oriented terminology, the following step-by-step process can be considered an instantiation of the SRP for Systems Development:

- Develop a schema based on observations and discussions with customers.
- Use the schema to make an inference by designing and developing a system functionality for a user requirement as understood using this schema.
- Test the inference by demonstrating the system to the users, observing users use the system, and discussing with them as to whether the system meets their requirements and work practices.

- If the test fails, there is a breakdown in understanding and the schema needs to be revised. The revised schema is created by (i) systems developers reflecting on why the break down occurred, (ii) determining what needs to be changed in the current schema to prevent the break down, and (iii) conducting new interviews and observations (if, after reflection and discussion, the reasons for the schema break down are unknown or unclear).
- If the test does not fail, the schema needs to be tested on other related strips. If all related tests do not fail, then the schema can be considered coherent. The system functionality designed and developed based on this coherent schema can be released to the users.

While this instantiation of SRP makes the steps involved in the process clearer for systems developers, it cannot be used in a mechanistic manner. The steps in this SRP instantiation that are firmly rooted in the ethnographic traditions and cannot be performed mechanistically are (a) observations, interviews, and discussions with users to test inferences in the form of designed and developed system functionalities and (b) reflections and discussions among systems developers to understand why breakdowns occurred and to revise the existing schema.

6 Applying the strip resolution process to determine user requirements

When the project was first proposed to us by one of the project sponsors, he defined the requirements informally in the following terms: "We have a poorly designed Microsoft Access database system for doing our work as stock analysts. This system is very slow. So, it is not used by any of the stock analysts. We would like to re-design the database and implement the database using a SQL Server database management system." When we conducted our first formal interviews with the two main project sponsors (the department head and a manager analyst), we tried to get behind the informal requirements stated in terms of technical infrastructure (database re-design and conversion from Microsoft Access to SQL Server) by focusing on two fundamental questions:

- Who are (or will be) the primary users of the system?
- What are their main work requirements?

During this first formal interview, we learned that the primary users of the system being developed were stock analysts. We also learned that one of the most important functions performed by a stock analyst was evaluating how much money an investment management company had invested in the stock of his client. The analyst called this

amount “stock holding.” Figure 2 shows the path we traveled using the SRP in understanding how stock analysts evaluated stock holdings. In the following sub-sections, we explain how our understanding of “evaluating stock holdings” progressed through each step of the SRP.

We were using the agile systems development approach of using short and iterative systems development cycles. As per one of the major tenets of agile development, we (the system developers and the customers) collaboratively prioritized the order in which we were going to develop system functionalities, starting with the most important. We keyed in on the functionality of enabling a stock analyst evaluate holdings as having the highest priority. As per a second major tenet of agile development, we (the system developers) decided to demonstrate and release functionalities in short cycles so that (a) the stock analysts could start benefiting from using the system and (b) we could identify problems and determine if the system met user requirements relatively quickly. Our agreement was that (i) we would meet with the project sponsors once every 2 weeks to demonstrate system functionalities we worked on during that period and (ii) we would release those functionalities that met user requirements into the production environment.

6.1 Schema-1: stock holdings can be evaluated ‘automatically’ by the system

After concluding our first formal interviews with the project sponsors, we (the two systems developers) went through our notes, reflected, discussed with each other, and developed our first schema (Schema-1) on how to evaluate stock holdings. Our understanding was that all the data elements required to calculate stock holdings were already in the database. Hence, our Schema-1 was “*Stock holdings can be evaluated ‘automatically’ by the system.*” Based on this schema, we wrote a stored procedure on the database to calculate stock holdings (Fig. 3).

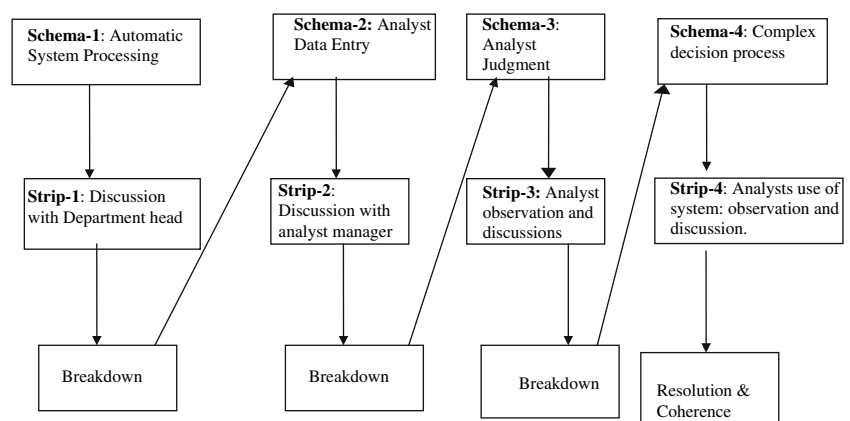
Two weeks later, we demonstrated how the system automatically calculated stock holdings to the department head. He stated that “automatic processing” did not represent how the stock holdings needed to be evaluated. He said that the aggregate value we had calculated (called “Aggregate Shares”) was just one of four factors that enabled a stock analyst to make a stock holding entry. The other three factors were (a) the amount of the last holding (the last value entered by the analyst), (b) a value obtained from a contact person at the Investment Management company, and (c) a publicly available value that had a 3-month delay called Schedule 13 F. He said that stock analysts had to make stock holding entries based upon considering all four factors. So, when we applied Schema-1 to the strip (Strip-1) represented by this discussion with the department head (i.e., we tested Schema-1 on Strip-1), we experienced a break down in our understanding (i.e., the test failed). The testing and breakdown of Schema-1 are shown boxed in Fig. 4.

6.2 Schema-2: evaluating holdings is “data entry”

We used what we learned from the meeting with the department head (which constituted the strip that led to our break down) combined with discussions and reflections (between the two of us, the system developers) on what led to the breakdown to revise our schema to “*Evaluating Holdings is a Data Entry process.*” This schema was our Schema-2. In this schema, our understanding was that an analyst evaluated and made the stock holdings entry using four values (a) last holding, (b) aggregate shares, (c) value obtained from Investment Management company contact person, and (d) value from imported Schedule 13 F file. Based on Schema-2, we revised our system design by creating a tabular structure for data entry (shown in Fig. 5).

Our Schema-2 resolved the breakdown on the first strip. However, for Schema-2 to be considered a coherent schema, it must enable us to interpret and understand

Fig. 2 Applying the strip resolution process: How do analysts evaluate stock holdings?



```

create proc TotalInvestmentManagerShares
    @IMID as integer,
    @ClientID as char(9)
as
    select Sum(AccountShares) as 'Total Holding'
    from ClientAccountSharesCAS, Account A
    where CASAccountNumber= A.AccountNumber
    and CASClientID= @ClientID
    and A.IMID = @IMID;

```

Total Holding = 15855000

Fig. 3 Schema-1: Evaluating stock holdings as automatic system processing

multiple related strips. A related strip was our discussion at our next fortnightly meeting with an analyst manager (this constitutes Strip-2 in Fig. 6) conducted when we showed him our revised design to enable the analyst to enter the stock holding value. During this discussion, the analyst manager told us that “stock analysts do not use a formula based on the four values [last holding, aggregate shares, investment manager contact value, Schedule 13 F value] to calculate an exact stock holding value.” He said that “the stock holding value entered was based on the analyst’s judgment [using the four values mentioned previously] and knowledge of the investment management company’s investment strategy [which the analyst, based on his knowledge of investment strategies, judges as Growth, Hedge, Contrarian, Quantitative, Indexed, etc].” Our Schema-2 broke down when tested on this new strip (Strip-2) of our discussion with an analyst manager. The testing and breakdown of Schema-2 are shown boxed in Fig. 6.

6.3 Schema-3: evaluating holdings requires “analyst’s judgment”

Our discussion with the analyst manager and reflections on the break down led to a revision of our existing schema

Aggregate Shares	15850000
From Schedule 13 F	17000000
Last Holding	14000000
Investment Manager Contact	16500000
Your Entry	
Submit	Submit

Fig. 5 Schema-2: Evaluate holdings as analyst data entry

(the revised schema was Schema-3). Schema-3 was that “Analysts evaluate stock holdings by making a judgment using their knowledge of the investment manager’s investment strategy.” One of the key factors that the analyst manager emphasized was that the stock holding evaluated by the analyst was not an exact value but a judgment. He said that analysis reports to clients would be much improved if the system allowed the analyst to provide a confidence level (a percentage that indicated how confident the analyst was with the stock holding value he entered) and explanatory notes on how the analyst arrived at his judgment. We applied our Schema-3 in revising the system design and functionality for evaluating stock holdings (shown in Fig. 7). We wanted to emphasize that evaluating stock holdings was not an exact science and involved analyst judgment. So, as requested by the manager analyst during our discussion, we provided a textbox for the confidence level indicator below the analyst’s estimate of stock holding. We also added a text area for notes for the analyst to explain how he made his judgment.

As with the previous Schemas, for Schema-3 to prove coherent, it must prevent break downs when tested on related strips. During our next visit to StockAnalysis Department, we took the system on a portable computer into the analysts’ work area intending to have two stock analysts use the re-designed functionality to test if it met their requirements. However, even before the analysts

Fig. 4 Breakdown of Schema-1: Evaluate holdings as automatic system processing

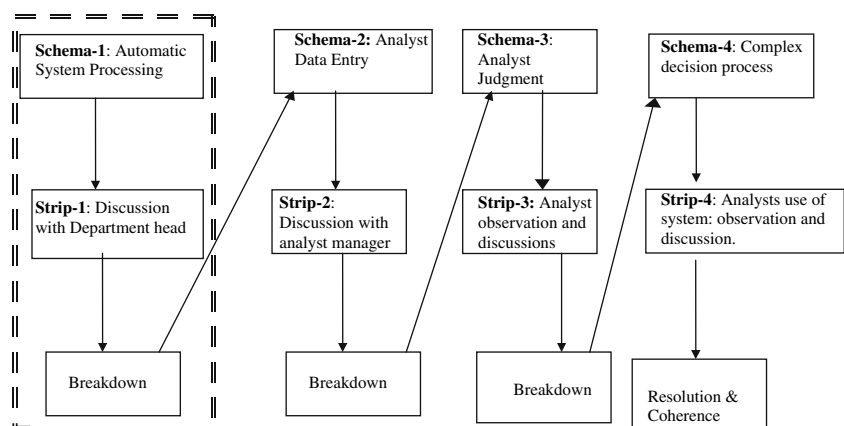
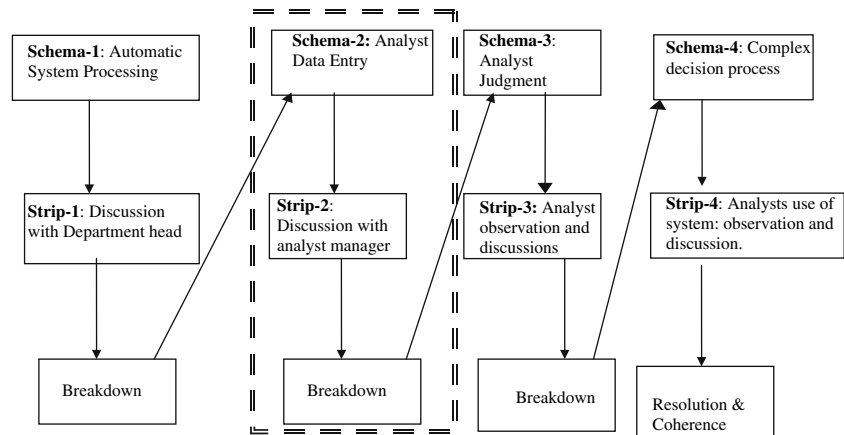


Fig. 6 Breakdown of Schema-2: Evaluate holdings as analyst data entry



Evaluate Holdings

Analyst: Hayes
it: Mens Wearhouse, Inc.
r: Cramer, Rosenthal & McGlynn, Inc.

Account Aggregation	15855000
From Schedule 13F	13500000
Last Holding:	14000000
Holding Date:	05/01/2001
Confidence Level	75%
Comments:	Account Aggregat
Investment Manager Contact:	Joe Smith
Position:	Manager
Telephone Number:	212-346-1000
Your Judgement:	
Confidence Level:	
Notes:	
Submit New Holding	Reset New Holding

Fig. 7 Schema-3: Evaluate holdings as analyst judgment

tested the re-designed functionality, we found our understanding of stock holding evaluation starting to fail. Schema-3 broke down when we observed the work and decision flow of the two stock analysts as they worked on evaluating stock holdings without using our system (these observations constituted the third strip, Strip-3). The stock analysts were using several documents (spread sheets, word documents, e-mail messages, faxed documents) when making their judgment. We saw that the analysts were cutting and pasting values from e-mail messages and word documents into spread sheets. They were calculating aggregates and generating graphs in the spread sheets. They were using a series of numbers spread over a period of time and across different investment management companies rather than just looking at the four values in Fig. 7 [which represented values for just one investment management company at one given point in time] before

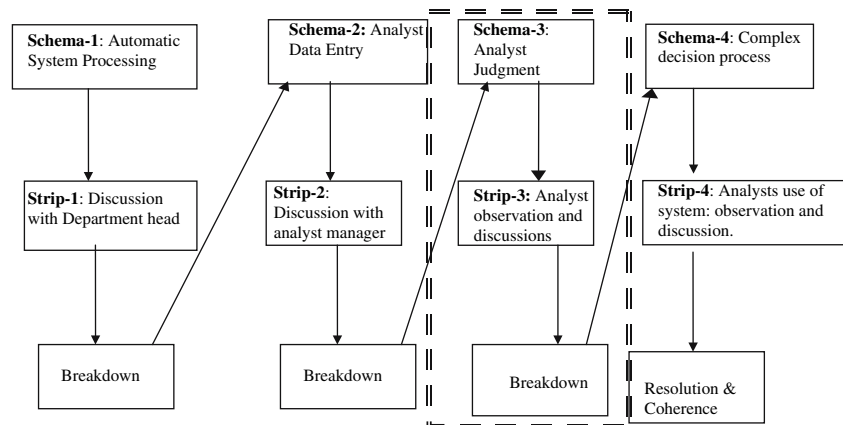
making their judgment. The testing and breakdown of Schema-3 are shown boxed in Fig. 8.

6.4 Schema-4: evaluating holdings is a “complex decision process”

The breakdown of Schema-3 was the most surprising breakdown for us as we had never heard about the use of historical and comparative data in evaluating stock holdings during the 2 months of meetings and discussions we had up until this point. In contrast, based on our retrospective reflections, we felt that our Schema-1 and Schema-2 breakdowns were less unexpected. Our Schema-1 had broken down because we had used our pre-existing background in database design and programming too heavily in coming up with an “automatic processing” schema. Our Schema-2 had broken down because we did not get sufficient detail about the stock holding evaluation process from the department head that one of the manager analysts was able to provide.

This unexpected breakdown of Schema-3 and the total lack of prior discussion about the use of historical and comparative data in evaluating stock holdings led us to the following conclusion. We concluded that (a) the different sources of data (Word documents, spreadsheets, e-mail, faxes, phone calls) from which analysts derived the aggregates and charts for historical and comparative analysis, and (b) the process they used to do derive these aggregates and charts (a combination of copying and pasting data into spreadsheets; entering data into spreadsheets from phone calls and faxes; using formulas to calculate aggregates; creating charts of these numbers and aggregates) were complex and not easily articulated. We determined that for us to understand this process we needed to conduct intensive interviews with the stock analysts in their natural work environment as they did their work of evaluating stock holdings [16, 17]. From these intensive interviews, we learned that two trend analysis indicators

Fig. 8 Breakdown of Schema-3: Evaluate holdings as analyst judgment



were missing in our system design. Stock analysts used a “historical trend analysis” of whether a particular investment management company was buying or selling their client’s stock over a period of time. They also used a “similar investors’ trend analysis” of whether investment management companies using similar investment strategies were buying or selling their client’s stock at a particular point in time. So, our newly revised schema (Schema-4) was that “*evaluating holdings is a complex decision process involving both comparative and historical trend analysis.*” We used Schema-4 to develop system functionalities represented in Figs. 9 and 10. Figure 9 represents the historical trend analysis of the last three stocks holdings of the investment management company (to enable the analyst determine the recent historical trends toward buying or selling of his client’s stocks and by what amounts). Figure 10 represents the buying or selling trends of “like-minded” investors presented in a new window

(to enable the analyst to understand what other investment management companies with similar investment strategies were doing).

Two weeks later, we tested Schema-4 on the related strip of having the stock analysts use the system with the functionalities in Figs. 9 and 10 to perform their stock analysis work. This strip (Strip-4) was the most rigorous test that we could apply to this schema since it tests if the schema can be used to design and develop a system that integrates with the work and decision flow of the stock analysts. In ethnographic terms, did our schema understand the natives’ (in this case, the stock analysts’) language, work and decision flow well enough to enable the system integrate with the work practices and requirements of the stock analysts? The analysts were able to use the system to perform their work meaning that Schema-4 did not break down. This implied that Strip-4 was resolved using Schema-4. The testing of Schema-4 on Strip-4 and the

Fig. 9 Schema-4: Evaluate holdings as complex decision process

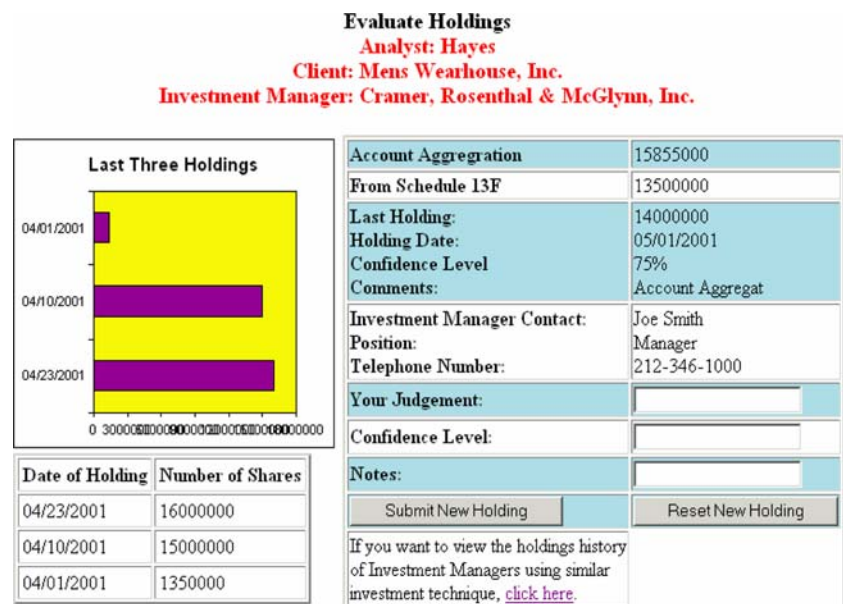


Fig. 10 Schema-4 (continued): Evaluate holdings as complex decision process



non-failure (resolution) of Schema-4 are shown boxed in Fig. 11.

We tested Schema-4 on other related strips. These related strips were observations and discussions with the stock analysts using the system on our subsequent visits. The stock analysts were able to continue using the system to perform their work of evaluating stock holdings. From these tests that did not fail, we concluded that Schema-4 was a coherent schema.

7 Discussion

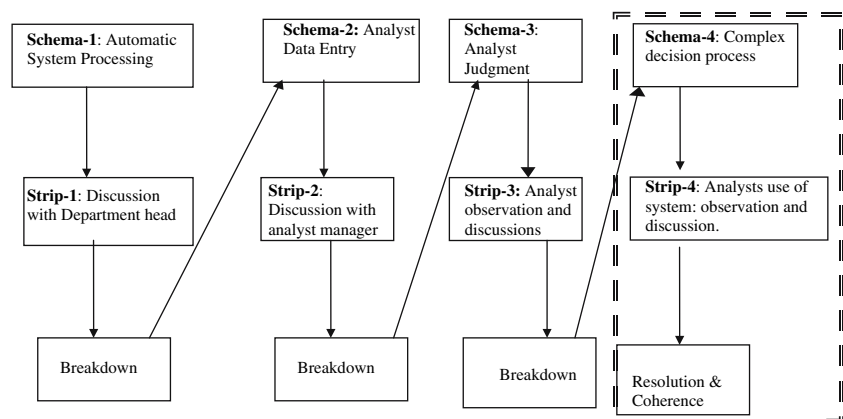
In this section, we discuss the lessons we learned and the insights we gained from applying the SRP to complement agile systems development at StockAnalysisDepartment. In particular, we discuss (a) the SRP serving as a bridge across the systems developer—user understanding gap, (b) the replicability of the SRP by other systems developers, (c) determining the validity of an interpretation using the

SRP, (d) how the SRP compares to other reflective learning processes, and (e) limitations of our study.

7.1 The strip resolution process as a bridge across the understanding gap

Agile development methodologies have gone beyond traditional (heavy) methodologies in emphasizing the importance of “soft” aspects of systems development such as communication, collaboration, and feedback. Agile methodologies also advocate accepting changing user requirements as inevitable instead of trying to fight or avoid such changes. However, even in agile methodologies, there exists an “understanding gap” between developers and users. Alistair Cockburn, a prominent agile development methodologist, stated the following regarding agile methodologies: “We are still in the infancy of naming what is really happening on software development projects. The answer is not process, modeling, or mathematics, although those play parts. [...] The next step is for methodologists to

Fig. 11 Resolved Schema-4: Evaluate holdings as complex decision process



partner with ethnographers, sociologists, and anthropologists to see if they have words to capture other parts of the [software development] experience” [9; p. 6].

Since the SRP is an ethnographic research process based on anthropological traditions, its’ emphasis is on understanding actors and their behaviors in their native environment. This emphasis could help close the “understanding gap” between developers and users (as illustrated in this study) and address what Cockburn sees as a current limitation of agile methodologies [9]. Hence, the SRP could be a good complement to agile development methods.

The project described in this paper applied agile development principles such as paired programming, frequent interaction with customers, short development and release cycles, and testing as a part of the development process. Our (the system developers) understanding of user requirements for this project was helped considerably by applying the SRP. The use case diagram in Fig. 12 represents our understanding of how stock holdings were evaluated prior to using the SRP. We identified only one use case, “Evaluate Holdings.” Using an agile methodology without the SRP, we might have been able to refine the use case diagram to better reflect the functional requirements in subsequent iterations. However, we may not have been able to understand and represent all the associated use cases (Fig. 13) without bridging the “understanding gap” between us in the role of system developers and the users. The SRP provides this bridge across the “understanding gap.” Also, the SRP provides a means of checking the validity of each use case by applying each use case “schema” on “strips” derived from observing and interviewing users.

An issue to consider is whether the SRP would be as useful as a bridge between the developer—user gap for a traditional systems development methodology as it is for an agile development methodology. For the SRP to be effective, it needs constant inference testing. For the systems developer, inference testing means developing, demonstrating, and releasing system functionalities to the users. Each inference is tested when the system functionalities are demonstrated and discussed with the users. A test fails when the users find that the functionality demonstrated

does not meet their requirements. Short systems development cycles of coding, demonstrating, and releasing system functionalities to the users are a fundamental characteristic of agile systems development. This characteristic makes the SRP a good complement for agile methodologies and, hence, useful as a bridge between the agile systems developer and user. However, the development cycles for traditional or heavy methodologies are significantly longer compared with agile methodologies resulting in much reduced inference testing. This characteristic makes the SRP less compatible with traditional or heavy methodologies and, hence, less useful as a bridge between the traditional systems developer and user.

7.2 Replicability of the strip resolution process

A fundamental question that arises, especially with an interpretive process such as the SRP, is whether another team of systems developers would have reached a similar coherent schema as we did if they had used the SRP in StockAnalysisDepartment.

An ethnography is an interpretation by the observer that depends upon the observer’s knowledge base and frames of references [1, 20]. Hence, the same observations can be interpreted differently by different observers. For example, the first schema we developed was “*Stock holdings can be evaluated ‘automatically’ by the system.*” Based on this schema, we wrote a stored procedure on the database to calculate stock holdings (Fig. 3). When we reflected on how we developed this schema (Schema-1), we surmised that it was probably influenced by (a) our strong background, educationally and professionally, in database design and programming and (b) the preliminary proposal of the sponsors stating the project objectives were database re-design and conversion from Access to SQL Server. Another systems developer with a strong background in object-oriented design might have developed a Schema-1 that was more object-oriented in nature.

This indicates that there might be differences in initial or intermediate schemas of two different systems development teams. However, an inherent strength of SRP, i.e., its’ iterative nature, leads toward replication of the final coherent schemas of the two teams. The SRP makes the systems developer conscious of the need to constantly iterate through the steps of (a) creating inferences based on his understanding of the users’ requirements (his schemas) by designing system functionalities, (b) testing those inferences by demonstrating the functionalities to users, and (c) reflecting on, and discussing, breakdowns in his understanding when some of those tests fail. This process leads the systems developer through a series of breakdowns and resolutions that ultimately lead him to a schema that is coherent. So, while the intermediate schemas in a SRP are

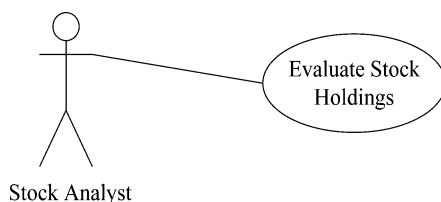
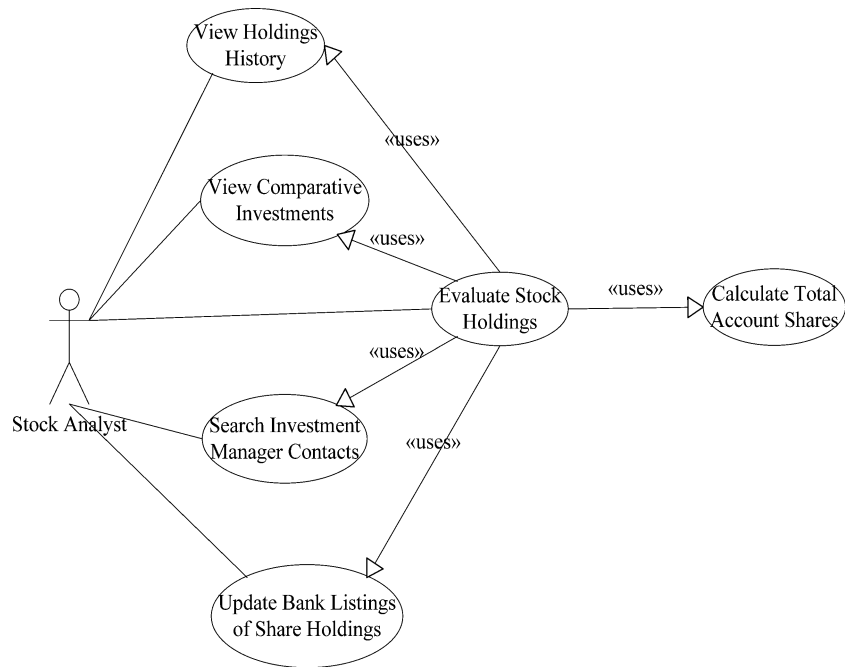


Fig. 12 Use case diagram: Before use of the SRP

Fig. 13 Use case diagram:
Complemented by the SRP



likely to vary from one developer to another both in terms of content of schemas and number of schemas, the final coherent schema of different systems developers for a given situation is likely to bear strong similarities.

7.3 Validity of an interpretation using the strip resolution process

Along with the issue of replicability, a second fundamental question that arises regarding an interpretive process such as the SRP is the validity of an interpretation, i.e., how to determine whether an interpretation is correct. A system developer's interpretation of users' requirements (developed using the SRP) can be validated in the following ways [1, 14, 18, 20, 21]:

(a) The developer should be able to communicate with the “natives,” i.e., the users, in their own context and in their own language. If the developer has understood the users, the developer should be able to communicate with the users about their work practices and requirements in the users' own work environment using the users' own terminology and vocabulary. If not, the system developer's interpretation of the users' requirements and work practices are not valid. An example in our project of communicating with the users in their own context and in their own language was when we went into the stock analysts' work area and observed and communicated with the analysts as they performed their work practices to evaluate stock holdings.

(b) The developer validates his interpretation by developing inferences (based on his interpretation) and testing them until he reaches an interpretation that does not breakdown (i.e., a coherent schema). In our system for StockAnalysisDepartment, we developed the “Evaluate Holdings” functionality by iteratively developing schemas, forming inferences by designing system functionalities, and testing the inferences by demonstrations and discussions. We released the “Evaluate Holdings” functionality when our tests did not fail and our schema did not breakdown. When we conducted our final discussions with the users several weeks after we released the “Evaluate Holdings” functionality, the users were able to continue using the system in their everyday work practices without a problem. These tests of our schema that did not fail indicate that our schema was coherent. A coherent schema suggests that we have reached a valid interpretation.

(c) Using development teams with members having different backgrounds can help validate an interpretation. Members with differing backgrounds are likely to have varying interpretations of user observations and discussions. This leads to discussion and reflection to reconcile varying interpretations. If variations in interpretations persist, they can be resolved by further observations of, and discussions with, the users. In our case, the two of us on the systems development team had extensive discussions after we observed and interviewed the department head, manager analysts, and the stock analysts (this

happened once every 2 weeks). Whenever one of us had a different interpretation, we noted down the differing interpretation to ask the relevant user to clarify. These discussions between us about our observations, and clarifying differences in our interpretations by collecting more data pertaining to those differences, ensured validity of our interpretation [26].

- (d) Using multiple sources of data helps validate an interpretation [28]. Yin calls this process “triangulating from multiple sources of evidence” [28, p. 99]. We used multiple sources of data (interviews, observations, documents) for our interpretations. A user’s statement might differ from what that user actually does in practice. We could validate whether the user’s statement was accurate by comparing his interview statements with his actions during our observations. Similarly, any observation that had ambiguous interpretations for us were verified by asking questions of the user to clarify the ambiguity.

7.4 Comparing the strip resolution process with other reflective learning processes

We used the SRP in this project since we drew upon ethnographic research approaches as exemplars to conduct our intensive field-based study. In its essence, the SRP is a self-reflective error detection and learning process. The SRP shares similarities with other organizational learning processes such as the “double-loop” learning concept developed by Argyris and Schön [4] or critical ethnography which uses the concept of the hermeneutic circle [14, 21]. So, learning processes similar to the SRP (such as double-loop learning or the hermeneutic circle) could potentially play the same role as the SRP in complementing projects applying agile methodology principles in helping determine uncertain and changing user requirements. However, as we discussed in the research method section, researchers using interpretive research methods spend several years learning how to apply interpretive processes. The SRP is a relatively systematic, step-by-step process to develop an interpretation and also includes terms to explain how this interpretation was developed. Most systems developers don’t have a pre-existing background in interpretive research approaches and cannot be expected to spend several years learning to apply these approaches. Hence, among the non-mechanistic error detection and learning processes that we reviewed (ethnographic processes, hermeneutics, and double-loop learning), the SRP is the most appropriate for use by systems developers. We have made it more readily usable for systems developers by creating an “agile systems

development instantiation” in the form of a non-mechanistic step-by-step SRP for systems development (discussed in Sect. 5).

7.5 Limitations

This study was conducted on a relatively small project with a team of two developers who developed the system by meeting with the customers once every 2 weeks in face-to-face sessions. From this study, we cannot generalize that the SRP will be similarly helpful in eliciting user requirements or adapting to changing user requirements on projects with different characteristics such as larger project scope, larger systems development teams, distributed development teams, etc.

Among the two developers in this study, one of us had attended university courses on doing ethnographic research in a graduate program and had applied the SRP on his doctoral dissertation research. So, we had some prior experience using the SRP for research, though not for systems development. Using the SRP following the step-by-step process we have provided in Sect. 5 should be feasible for most systems developers. However, we need to study other teams of agile systems developers using the SRP to determine its ease, use, and relevance for agile systems development.

8 Conclusion

Agile methodologies emphasize the importance of “soft” aspects such as communication, collaboration, and feedback in addition to the technical aspects such as technology, process, tools, notation, and modeling for effective systems development. However, there is a continuing “understanding gap” between developers and customers even when using agile methodologies to develop systems [9].

The ethnographic process of Strip Resolution has been used in anthropological studies [1] as a “researcher’s tool” to better understand people from societies different from the researchers’. In this study, we used the SRP to reduce the “understanding gap” between us, in the role of systems developers, and the stock analysts, analyst managers, and department director who were our customers. We used this process to understand the evolving requirements of stock analysts to design and develop a system that supported the analyst’s work and decision flows in evaluating “stock holdings.” This study illustrates that the SRP could be used as a systematic process to complement projects applying agile methodology principles in determining and adapting to evolving user requirements and in effectively incorporating customers into the agile development process.

References

1. M. Agar, *Speaking of Ethnography* (Sage Publications, Beverly Hills, CA, 1986)
2. Agile Alliance, Manifesto for agile software development, <http://www.agilealliance.org/resources/roadmap/> (2001)
3. S. Ambler, *Agile Modeling: Effective Practices for eXtreme Programming and the Unified Process* (John Wiley & Sons, New York, NY, 2002)
4. C. Argyris, D. Schön, *Organizational Learning: A Theory of Action Perspective* (Addison Wesley, Reading, MA, 1978)
5. K. Auer, R. Miller, *Extreme Programming Applied: Playing to Win* (Addison-Wesley, New York, NY, 2002)
6. D. Avison, G. Fitzgerald, *Commun. ACM* **46**(1), 79 (2003)
7. K. Beck, C. Andres, *Extreme Programming Explained: Embrace Change*, 2nd edn. (Addison-Wesley, New York, NY, 2005)
8. R. Casson, *Ann. Rev. Anthropol.* **12**, 429 (1983)
9. A. Cockburn, *Agile Software Development* (Addison-Wesley, New York, NY, 2002)
10. A. Cockburn, *Crystal Clear: A Human-Powered Methodology for Small Teams* (Pearson Education Inc., Upper Saddle River, NJ, 2005)
11. J. Erickson, K. Lyytinen, K. Siau, *J. Database. Manage.* **16**(4), 88 (2005)
12. M. Fowler, The new methodology, <http://www.martinfowler.com/articles/newMethodology.html> (2005)
13. J. George, D. Batra, J. Valacich, J. Hoffer, *Object-Oriented Systems Analysis and Design* (Pearson Education, Upper Saddle River, NJ, 2004)
14. L. Harvey, M. Myers, *Inform. Technol. People* **8**(3), 13 (1995)
15. J. Highsmith, *Agile Software Development Ecosystems* (Addison-Wesley, New York, NY, 2002)
16. K. Holzblatt, H. Beyer, *Commun. ACM* **36**(10), 93 (1993)
17. J. Hughes, D. Randall, D. Shapiro, in *Proceedings of the Conference on Computer-supported Co-operative Work: Sharing Perspectives*, ACM Press, New York, NY, 1992
18. H. Klein, M. Myers, *MIS Quart.* **23**(1), 67 (1999)
19. A. Lee, *MIS Quart.* **13**(1), 33 (1989)
20. M. Myers, *Commun. AIS* **2**(23), 1 (1999)
21. M. Myers, L. Young, *Inform. Technol. People.* **10**(3), 224 (1997)
22. J. Nandhakumar, D. Avison, *Inform. Technol. People* **12**(2), 176 (1999)
23. W. Orlikowski, in *Proceedings of the Tenth International Conference on Information Systems*, Boston, MA, 1989
24. U. Schultze, *MIS Quart.* **24**(1), 1 (2000)
25. K. Schwaber, *Agile Project Management with Scrum* (Microsoft Press, Redmond, WA, 2004)
26. E. Trauth, L. Jessup, *MIS Quart.* **24**(1), 43 (2000)
27. D. Turk, R. France, B. Rumpe, *J. Database Manage.* **16**(4), 62 (2005)
28. R. Yin, *Case Study Research: Design and Methods* (Sage Publications, Thousand Oaks, CA, 1994)
29. S. Zuboff, in *The Age of the Smart Machine: The Future of Work and Power* (Basic Books 1988)